Анализ данных с использованием языка Python

Пояснения к коду. Семинар 5

Pandas: фильтрация, сортировка, группировка, агрегация, сводные таблицы

А. Н. Лата

Пояснения к ноутбуку `notebooks/seminar-05-pandas-groupby.ipynb`. Номер ячейки — её порядковый номер среди ячеек кода ноутбука, считая сверху. Для каждой ячейки приведён её код с номерами строк, а под ним — что делает каждая команда и что означает каждый аргумент именно в этой ячейке. Команды, разобранные в пояснениях к семинарам 1–4, здесь только называются. Код совпадает с ноутбуком, только пробелы перед комментариями сокращены. Полные подписи функций со всеми параметрами — в справочнике `reference/pandas.md`.

1 Вопросы, на которые маска не отвечает

Ячейка 1. Подключение pandas

```
1 # Импортируем библиотеку pandas
2 import pandas as pd
```

– `import pandas as pd`, как на семинаре 4.

Ячейка 2. Загрузка таблицы

```
1 # Загрузка данных о ДТП в Брянской области из CSV-файла
2 dtp = pd.read_csv('../datasets/legal/dtp-bryansk.csv', index_col='id')  # ↩
  ↪ ДТП Брянской области
3 dtp.head(3)  # первые три ДТП
```

– `pd.read_csv(..., index_col='id')` и `head(3)` — как на семинаре 4.

Ячейка 3. Один район маской

```
1 pochep = dtp['district'] == 'Почепский район'  # маска: Почепский район
2 pochep.sum(), (dtp.loc[pochep, 'dead_count'] > 0).sum()  # ДТП и ДТП с ↩
  ↪ погибшими
```

– `pochep.sum()` — число ДТП района. `(dtp.loc[pochep, 'dead_count'] > 0).sum()` — погибшие только в этом районе, маска «больше нуля» и её сумма: число ДТП с погибшими.

2 Фильтрация: query и строки

Ячейка 4. Условие текстом

```
severe_day = dtp.query('dead_count >= 3 and light == "Светлое время ↩
↪ суток"')  # три и более погибших днём
severe_day[['datetime', 'district', 'category', 'dead_count']].head()
```

– `dtp.query('...')` — отбор строк по условию, записанному строкой. Имена столбцов пишутся как есть, условия соединяет `and`, скобки вокруг сравнений не нужны.

– `"Светлое время суток"` — строка внутри строки: снаружи одинарные кавычки, внутри двойные.

– `severe_day[[...]].head()` — первые пять строк результата, только нужные столбцы.

Ячейка 5. Сколько строк

```
len(severe_day)  # число таких ДТП
```

– `len(severe_day)` — число строк таблицы.

Ячейка 6. Переменная в условии

```
limit = 10  # порог по раненым
dtp.query('injured_count >= @limit')[['datetime', 'district', 'category', ↩
↪ 'injured_count']]
```

– `@limit` — знак `@` перед именем подставляет в условие значение переменной Python.

Ячейка 7. Поиск подстроки

```
night = dtp['light'].str.contains('темное')  # маска: ночные ДТП
night.sum()
```

– `dtp['light'].str` — доступ к строковым методам столбца; `.contains('темное')` — маска: есть ли подстрока в значении.

Ячейка 8. Начало строки

```
in_bryansk = dtp['district'].str.startswith('Брянск')  # Брянская область ↩
↪ и Брянский район
in_bryansk.sum(), (~in_bryansk).sum()
```

– `str.startswith('Брянск')` — маска: начинается ли значение с этих символов.

– `(~in_bryansk).sum()` — число остальных строк.

Ячейка 9. Буква ё: тихая ловушка

```
dtp['light'].str.contains('тёмное').sum()  # с буквой ё: ноль совпадений
```

– `str.contains('тёмное')` — поиск чувствителен к каждой букве: «ё» и «е» разные, и совпадений ноль. Ошибки при этом нет.

3 Сортировка

Ячейка 10. По одному столбцу

```
fields = ['datetime', 'district', 'category', 'dead_count', 'injured_count']  # поля для отчёта
dtp.sort_values('dead_count', ascending=False)[fields].head()
```

– `fields = [...]` — список столбцов для показа, задан один раз.

– `dtp.sort_values('dead_count', ascending=False)` — таблица, отсортированная по числу погибших; `ascending=False` — по убыванию.

– `[fields].head()` — из отсортированной таблицы только эти столбцы и первые пять строк. Цепочка читается слева направо.

Ячейка 11. По двум столбцам

```
dtp.sort_values(['dead_count', 'injured_count'], ascending=False)[fields].head()
```

– `sort_values(['dead_count', 'injured_count'], ...)` — список столбцов: сначала по первому, а при равенстве — по второму.

Ячейка 12. Наибольшие

```
dtp.nlargest(3, 'injured_count')[fields]  # три ДТП по раненым
```

– `dtp.nlargest(3, 'injured_count')` — три строки с наибольшими значениями столбца, по убыванию. Таблица целиком не сортируется.

Ячейка 13. Сортировка без присваивания

```
dtp.sort_values('dead_count', ascending=False)  # сортировка без присваивания
dtp.head(2)  # таблица прежняя
```

– Строка 1 — отсортированная таблица создаётся и никуда не записывается. Последняя строка ячейки — `dtp.head(2)`, поэтому на экране прежний порядок.

Ячейка 14. Отсортированная копия

```
by_dead = dtp.sort_values('dead_count', ascending=False)  # ↩
↪ отсортированная копия
by_dead.head(2)[fields]
```

– `by_dead = dtp.sort_values(...)` — результат записан в новую переменную; `dtp` не меняется.

4 Новые столбцы

Ячейка 15. Столбец из выражения

```
dtp['victims'] = dtp['dead_count'] + dtp['injured_count']  # пострадавшие: ↩
↪ погибшие и раненые
dtp[['dead_count', 'injured_count', 'victims']].head(3)
```

– `dtp['victims'] = ...` — присваивание по новому имени создаёт столбец. Сумма двух столбцов считается построчно.

Ячейка 16. Год срезом строки

```
dtp['year'] = dtp['datetime'].str[:4]  # год ДТП строкой
dtp['year'].value_counts().sort_index()  # ДТП по годам
```

– `dtp['datetime'].str[:4]` — первые четыре символа каждой строки даты: год, тоже строкой.

– `value_counts().sort_index()` — частоты, упорядоченные по меткам, то есть по годам, а не по убыванию числа.

Ячейка 17. Замена по словарю

```
daytime = {'Светлое время суток': 'день',
           'Сумерки': 'сумерки',
           'В темное время суток, освещение включено': 'ночь',
           'В темное время суток, освещение отсутствует': 'ночь',
           'В темное время суток, освещение не включено': 'ночь'}  # ↩
↪ правило замены
dtp['daytime'] = dtp['light'].map(daytime)  # время суток из трёх значений
dtp['daytime'].value_counts()
```

– Строки 1–5 — словарь: ключ — значение освещения из данных, значение — новая подпись.

– `dtp['light'].map(daytime)` — каждое значение столбца заменяется по словарю. Значение, которого нет среди ключей, становится пропуском.

Ячейка 18. Своя функция

```
1  def quarter(month):
2      if month in ('01', '02', '03'):
3          return 'I'
4      if month in ('04', '05', '06'):
5          return 'II'
6      if month in ('07', '08', '09'):
7          return 'III'
8      return 'IV'
9
10
11 dtp['quarter'] = dtp['datetime'].str[5:7].apply(quarter)  # квартал по месяцу
12 dtp['quarter'].value_counts().sort_index()
```

– Строки 1–8 — функция `quarter`: если месяц входит в набор `('01', '02', '03')`, вернуть `'I'`, и так далее; `in` — проверка вхождения. До последнего `return` доходит только месяц четвёртого квартала.

– `dtp['datetime'].str[5:7]` — шестой и седьмой символы даты: месяц строкой от `'01'` до `'12'`.

– `.apply(quarter)` — вызвать функцию для каждого значения и собрать результаты в столбец. Имя функции передаётся без скобок.

Ячейка 19. Запись в копию: намеренное предупреждение

```
1  dtp[dtp['district'] == 'Брянская область']['district'] = 'без района'  # ↩
   ↪ запись в копию: не работает
2  dtp['district'].value_counts().head(3)  # подписи прежние
```

– `dtp[маска]['district'] = ...` — сначала `dtp[маска]` создаёт временную таблицу, потом запись идёт в неё. Таблица `dtp` не меняется; pandas выводит предупреждение `ChainedAssignmentError`.

Ячейка 20. Запись по маске через loc

```
1  dtp.loc[dtp['district'] == 'Брянская область', 'district'] = 'без района'  ↩
   ↪ # запись через loc
2  dtp['district'].value_counts().head(3)  # подпись заменена
```

– `dtp.loc[маска, 'district'] = 'без района'` — маска строк и столбец в одних скобках: запись идёт в саму таблицу, только в отмеченные строки.

5 Группировка

Ячейка 21. Сумма по районам

```
dtp.groupby('district')['dead_count'].sum().sort_values(ascending=False).↩
↪ head()  # погибшие по районам
```

– `dtp.groupby('district')` — разделить таблицу на группы по району. `['dead_count']` — столбец, по которому считается статистика; `.sum()` — сумма в каждой группе.

– Результат — Series: названия районов в индексе. Дальше `sort_values` по убыванию и пять первых.

Ячейка 22. Несколько статистик

```
dtp['with_dead'] = dtp['dead_count'] > 0  # маска как столбец
dtp.groupby('category')['with_dead'].agg(['size', ↩
↪ 'mean']).sort_values('mean', ascending=False).round(3).head()
```

– `dtp['with_dead'] = ...` — маска, записанная столбцом: в каждой строке `True` или `False`.

– `.agg(['size', 'mean'])` — две статистики в каждой группе: `size` — число строк, `mean` логического столбца — доля `True`. Результат — таблица из двух столбцов.

– `.sort_values('mean', ascending=False)` — у таблицы сортировка по имени столбца.

Ячейка 23. Именованные статистики

```
by_year = dtp.groupby('year').agg(accidents=('dead_count', 'size'),
                                  dead=('dead_count', 'sum'),
                                  injured=('injured_count', 'sum'))  # ↩
↪ сводка по годам
by_year
```

– `agg(accidents=('dead_count', 'size'), ...)` — слева имя столбца результата, справа пара: по какому столбцу и какую статистику считать. Вызов записан на трёх строках.

Ячейка 24. Series по Series

```
night.groupby(dtp['year']).mean().round(3)  # доля ночных ДТП по годам
```

– `night.groupby(dtp['year'])` — группируется не таблица, а маска `night` из ячейки 7, по значениям другого столбца той же длины. Строки сопоставляются по индексу.

Ячейка 25. Среднее по всей группировке: намеренная ошибка

```
dtp.groupby('district').mean()  # среднее по тексту: ошибка
```

– Без выбора столбца `mean()` считается по всем столбцам, и на текстовом падает: `TypeError`.

Ячейка 26. Список столбцов

```
dtp.groupby('district')[['dead_count', 'injured_count']].mean().round(2).head(3)  # средние по числовым столбцам
```

– `[['dead_count', 'injured_count']]` — список столбцов после `groupby`: статистика только по ним, результат — таблица.

6 Сводные таблицы

Ячейка 27. Сводная таблица

```
pd.pivot_table(dtp, values='dead_count', index='year', columns='daytime',
               aggfunc='sum', margins=True)  # погибшие: год и время суток
```

– `pd.pivot_table(dtp, values=..., index=..., columns=..., aggfunc=...)` — `values` — столбец, по которому считается статистика; `index` — что по строкам; `columns` — что по столбцам; `aggfunc='sum'` — какая статистика.

– `margins=True` — итоговые строка и столбец с именем `All`.

Ячейка 28. Пропуски в сводной

```
pd.pivot_table(dtp, values='dead_count', index='quarter', columns='weather',
               aggfunc='sum')  # погибшие: квартал и погода
```

– Сочетание квартала и погоды, которого нет в данных, даёт `NaN`; из-за пропусков числа показаны дробными.

Ячейка 29. Нули вместо пропусков

```
pd.pivot_table(dtp, values='dead_count', index='quarter', columns='weather',
               aggfunc='sum', fill_value=0, margins=True)  # то же с нулями и итогами
```

– `fill_value=0` — пустые клетки заполняются нулём, и числа снова целые.

Ячейка 30. Таблица сопряжённости

```
kinds = ['Столкновение', 'Наезд на пешехода', 'Съезд с дороги']  # три массовых вида ДТП
pd.crosstab(dtp['category'], dtp['severity']).loc[kinds]  # ДТП по виду и тяжести
```

– `pd.crosstab(dtp['category'], dtp['severity'])` — число строк для каждого сочетания двух столбцов; столбец значений не нужен.

– `.loc[kinds]` — из результата только строки трёх видов из списка, в его порядке.

Ячейка 31. Доли внутри строки

```
pd.crosstab(dtp['category'], dtp['severity'], normalize='index').round(3).loc[kinds]  # доли внутри вида
```

– `normalize='index'` — каждая строка делится на свою сумму: доли тяжестей внутри вида ДТП, в строке в сумме единица.

7 Мини-анализ и итоги

Ячейка 32. Новый столбец сводки

```
by_year['fatal_share'] = dtp.groupby('year')['with_dead'].mean().round(3)  # доля ДТП с погибшими
by_year
```

– `by_year['fatal_share'] = ...` — столбец сводки из другой группировки; значения встают по совпадающим меткам индекса, то есть по годам.

Ячейка 33. Годы и тяжесть

```
pd.crosstab(dtp['year'], dtp['severity'])[['Легкий', 'Тяжёлый', 'С погибшими']]  # ДТП по годам и тяжести
```

– `pd.crosstab(dtp['year'], dtp['severity'])` — число ДТП по годам и тяжести; `[[...]]` — три столбца в нужном порядке.

Ячейка 34. Районы от ста ДТП

```
by_district = dtp.groupby('district').agg(accidents=('with_dead', 'size'),
                                          fatal_share=('with_dead', 'mean'))  # ДТП и доля с погибшими
large = by_district[by_district['accidents'] >= 100]  # районы от 100 ДТП
large.sort_values('fatal_share', ascending=False).round(3).head()
```

– Строки 1–2 — именованные статистики: число ДТП и доля с погибшими по районам.

– `by_district[by_district['accidents'] >= 100]` — маска по столбцу сводки: оставить районы, где ДТП не меньше ста.

Ячейка 35. По погоде

```
dtp.groupby('weather')['with_dead'].agg(['size', 'mean']).round(3)  # по погоде: ДТП и доля
```

– `agg(['size', 'mean'])` по погоде, как в ячейке 22: число ДТП и доля с погибшими рядом.

Ячейка 36. Сводка в файл

```
report = large.sort_values('fatal_share', ascending=False).round(3)
report = report.rename(columns={'accidents': 'ДТП', 'fatal_share': 'доля с погибшими'})  # подписи для отчёта
report.to_csv('dtp-bryansk-districts.csv')  # сводка в файл
report.head(3)
```

– `report.rename(columns={'accidents': 'ДТП', ...})` — переименовать столбцы по словарю «старое имя: новое»; результат — новая таблица.

– `report.to_csv(...)` — сводка в файл; индекс с названиями районов — первым столбцом.